Chapitre 3
Diffusion

Notes rédigées par Raphaël JONTEF

cours dispensé par M. Cyril Gavoille

Enseirb-Matmeca
filière informatique

14 octobre 2026

Table des matières

I Introduction **2**

II Arbre de diffusion **2**

II.1 Message *vs.* connaissance 3

I Introduction

Supposons vouloir envoyer depuis un sommet distingué r_0 le même **petit** message M à tous les autres sommets d'un graphe $G = (V, E)$.

Proposition 3.1

Pour tout graphe G connexe et sommet distingué r_0, tout algorithme distribué A de diffusion sur G et quelle que soit la connaissance à priori, on a :

— $\text{MESSAGE}(A, G) \geq n - 1$

— $\text{TEMPS}(A, G) \geq \text{ecc}_G(r_0)$

II Arbre de diffusion

On suppose qu'on dispose d'un arbre couvrant T enraciné en r_0 donné par $\begin{cases} u \mapsto \text{parent}(u) \\ u \mapsto \text{Enfants}(u) \end{cases}$ On considre l'algorithme de diffusion suivant :

```
1 CAST(T, r0):
2   Si u ≠ r0:
3     M := RECEIVE()
4   Pour tout v dans Enfants(u):
5     SEND(M,v)
```

Proposition 3.2 - *propriétés de cet algorithme*

— $\text{TEMPS}\Big(CAST(T, r_0), G\Big) = \text{ecc}_T(r_0) = \text{hauteur}(T)$

— $\text{MESSAGE}\Big(CAST(T, r_0), G\Big) = |E(T)| = n - 1$

Cet algorithme a le mérite de limiter le nombre de messages à $n - 1$ mais il nécessite d'avoir T. Considérons à présent l'algorithme asynchrone suivant :

```
1 FLOOD(T, r0):
2 si u = r0:
3   SENDALL(M) // envoi à tous les voisins
4 sinon:
5   M := receive()
6   v := émetteru du message M
7   SEND(M, w) pour tout w ∈ Voisins(u) \ {v
```

Proposition 3.3 - *propriétés de cet algorithme*

On a, en notant m le nombre d'arêtes de G, et n son nombre de sommets :

— $\text{MESSAGE}\Big(FLOOD(r_0), G\Big) = m = \mathcal{O}(n^2)$

— $\text{TEMPS}\Big(FLOOD(r_0), G\Big) = \text{ecc}_G(r_0)$

Démonstration

1.
$$\begin{aligned}\text{MESSAGE}\Big(FLOOD(r_0), G\Big) &= \deg_G(r_0) \sum_{\substack{u\in V\\ u\neq r_0}} \Big(\deg_G(u) - 1\Big)\\ &= \sum_{u\in V} \deg(u) - (n-1)\\ &= 2m - n + 1\end{aligned}$$

2. Montre qu'après t unités de temps, tous les sommets à distances $\leq t$ ont reçu M. Il y a donc au plus $\text{ecc}_G(r_0)$ unités de temps avant que tous les sommets aient reçu M.

Il existe également une variante synchrone "amnésique" de FLOOD qui consiste à appliquer FLOOD perpétuellement.

```
AMNESIAC-FLOOD(r0):
si u = r0:
 SENDALL(M)
S = ∅
tant que vrai:
 démarrer une nouvelle ronde
 si S = ∅:
   SEND(M, v) pour tout v ∈ Voisins(u) \ S
   // c'est grâce à ça qu'on s'arrête
   M := RECEIVE_ALL() // on reçoit tous les messages des voisins
   S := l'ensemble des émetteurs de M
   si S ≠ ∅:
     choisir M ∈ M
```

II.1 Message *vs.* connaissance

	TEMPS	MESSAGE	"CONNAISSANCE"
CAST_T	$\text{ecc}_T(r_0)$	$n-1 = \mathcal{O}(n)$	T
FLOOD	$\text{ecc}_G(r_0)$	$2m - n + 1 = \mathcal{O}(n^2)$	$\varnothing$

Définition 3.4 - *connaissance à distance ρ*

On dit qu'un sommet u a connaissance à distance ρ si u connaît l'ensemble des sommets à distance $\leq \rho$ de lui, ainsi que les arêtes entre ces sommets.

Proposition 3.5 - *petits messages*

$\forall \rho \in \mathbb{N}, \forall G$ connexe à n sommets et r_0, il existe un algorithme de diffusion FLOODK_ρ avec une connaissance ρ tel que :

$$\text{MESSAGE}\Big(\text{FLOODK}_\rho(r_0), G\Big) = 2\min(m, n^{1+\frac{1}{\rho}})$$

Ainsi :

— FLOODK_0 est en $\mathcal{O}(n^2)$

— FLOODK_1 est l'algorithme FLOOD, également en $\mathcal{O}(n^2)$ qui ne nécessite aucune connaissance.

— FLOODK_2 est en $\mathcal{O}(n^{3/2})$

— FLOODK_3 est en $\mathcal{O}(n^{4/3})$

— FLOODK_∞ est l'algorithme CAST_T, en $\mathcal{O}(n)$

Démonstration

On prend $\rho \geq 1$. Deux idées :

— Localement, chaque sommet décide de désactiver certaines arêtes, et $\tilde{G}$ le graphe résultant.

— On applique FLOOD dans $\tilde{G}$, $\text{MSG} \leq 2\tilde{m} - n + 1$

On associe un identifiant $\text{ID}(u, v)$ à chaque arête (u, v). On impose la contrainte suivante :

Proposition 3.6

u désactive $u - v$ s'il existe dans G un cycle $\mathcal{C}$ de longueur $\leq 2\rho$ contenant $u - v$, d'identifiant minimal dans ce cycle.

On parle ici de cycles au sens où l'on ne passe pas plusieurs fois par la même arête.

Remarque 3.7

Chaque sommet peut appliquer la règle localement, mais pas sur le même cycle. On ne peut pas être facilement sûr que le graphe d'arrivée soit connexe, sachant que toutes les arêtes sont enlevées non pas successivement, mais simultanément.

Théorème 3.8

Si G est connexe, alors $\tilde{G}$ est connexe.

Démonstration

On montre le résultat par récurrence sur l'indice de l'arête supprimée (on les ordonne arbitrairement). On note G_i le graphe après suppression des i premières arêtes. On suppose que G_i est connexe et on montre que G_{i+1} est connexe.

On note $e = (u, v)$ l'arête supprimée à l'étape $i + 1$. Par hypothèse, il existe un cycle $\mathcal{C}$ de longueur $\leq 2\rho$ contenant e. Comme $\mathcal{C}$ est un cycle, il existe un chemin de u à v dans $\mathcal{C} \setminus \{e\}$. Ce chemin est contenu dans G_i, donc u et v sont encore connectés dans G_{i+1}. Ainsi, la suppression de l'arête ne déconnecte pas le graphe.

On peut se questionner sur le majorant 2ρ. Il découle du résultat suivant :

Théorème 3.9

Si G n'a pas de cycle de longueur $\leq 2\rho$, alors il a au plus $n^{1+\frac{1}{\rho}} + n$

Démonstration

On note $d = \frac{m}{n}$ le degré moyen. On supprime tous les sommets de degré $< d$ de G. Ainsi, on obtient un sous-graphe H de G, qui contient au moins un arbre $\mathcal{A}$ complet d'arité d, et de profondeur ρ. On compte les feuilles de cet arbre $\mathcal{A}$: il y en a $(d-1)^\rho$. Or, ces feuilles sont des sommets de H, donc de G. On a alors :

$$n \geq (d-1)^\rho$$

Donc :

$$n \geq \left(\frac{m}{n} - 1\right)^\rho$$

D'où le résultat.